

Lookup Table-based Multiplication-free All-digital DNN Accelerator Featuring Self-Synchronous Pipeline Accumulation

Hiroto Tagata
Graduate School of Informatics
Kyoto University
Kyoto, Japan
htagata@easter.kuee.kyoto-u.ac.jp

Takashi Sato
Graduate School of Informatics
Kyoto University
Kyoto, Japan
takashi@i.kyoto-u.ac.jp

Hiromitsu Awano
Graduate School of Informatics
Kyoto University
Kyoto, Japan
awano@i.kyoto-u.ac.jp

Abstract—Deep neural networks (DNNs) have been widely applied in our society, yet reducing power consumption due to large-scale matrix computations remains a critical challenge. MADDNESS is a known approach to improving energy efficiency by substituting matrix multiplication with table lookup operations. Previous research has employed large analog computing circuits to convert inputs into LUT addresses, which presents challenges to area efficiency and computational accuracy. This paper proposes a novel MADDNESS-based all-digital accelerator featuring a self-synchronous pipeline accumulator, resulting in a compact, energy-efficient, and PVT-invariant computation. Post-layout simulation using a commercial 22nm process showed that $2.5\times$ higher energy efficiency (174 TOPS/W) and $5\times$ higher area efficiency (2.01 TOPS/mm²) can be achieved compared to the conventional accelerator.

Index Terms—Approximate Matrix Multiplication, Computing-in-Memory, Convolutional Neural Network, SRAM, Binary Decision Tree.

I. INTRODUCTION

Deep neural networks (DNNs) are now widely applied in various fields, such as pattern recognition and natural language processing [1]–[3]. As their performance advances, DNNs continue to expand in parameter size[4], resulting in increased inference costs, mainly due to the high computational demands of multiply-and-accumulate (MAC) operations. In particular, the memory access needed to retrieve large amounts of weight parameters during inference is a significant factor in increasing energy consumption and latency [5].

Computing-in-Memory (CIM) is a leading approach to reducing memory access costs by integrating computation circuits directly into memory, thereby enhancing inference efficiency [6]. CIM is categorized into two types: analog and digital. Analog CIM employs voltage, current, and delay for computation, enabling high-speed, energy-efficient MAC operations without relying on advanced process nodes [7]. However, its inference accuracy is susceptible to process, voltage, and temperature (PVT) variations, and the increasing demand for multi-bit input/output support has introduced substantial costs associated with DA/AD conversion [8], [9]. In contrast, digital CIM offers high precision and robustness against PVT variations, with the potential for enhanced power, performance, and

area (PPA) through scaling [10], [11]. However, it relies on large multipliers and adders, constraining energy efficiency. In particular, multipliers consume 6 to 31 times more energy and occupy 8 to 25 times more area than adders [12], [13].

One solution to reduce the high computational cost of multipliers is approximate matrix multiplication (AMM) using lookup tables (LUT) [14]–[18]. Especially, MADDNESS (Multiply-ADDitioN-IeSS)[16] employs a product quantization (PQ) technique[19], whereby the input activation is approximated by prototypes that have been learned prior to the inference. Since the multiplication of the prototypes and known synaptic weight can be computed offline, the precomputed results can be stored in LUT. This approach enables matrix multiplication through low-cost table lookup operations, significantly reducing computational energy. Although PQ introduces some accuracy loss, previous studies report that 16 or more prototypes can achieve accuracy comparable to conventional DNNs [16]–[18].

Nevertheless, the implementation of MADDNESS in hardware presents significant challenges. The computational costs associated with MADDNESS are dominated by the PQ and data readout cost from LUT. However, GPU lacks optimization for these operations, limiting both throughput and energy efficiency [20]. While some MADDNESS accelerators have been proposed, challenges remain: for example, [21] uses analog computation in PQ, making it vulnerable to accuracy degradation from PVT variations. Furthermore, the synthesizable accelerator proposed in [22] has high LUT read energy attributed to the redundant layout, reducing energy efficiency.

This paper proposes a MADDNESS-based all-digital accelerator featuring a self-synchronous pipeline accumulator. The proposed circuit employs a data-driven asynchronous design, eliminating the need for a global clock and providing robust PVT variation tolerance. Furthermore, it incorporates a dataflow-based decision tree classifier for the PQ circuit and an area-efficient memory using 10-transistor static random access memory (10T-SRAM), achieving substantial reductions in energy consumption compared to the previous design.

The contributions of this paper are as follows:

- This study introduces the first compact, energy-efficient, and PVT-invariant MADDNESS accelerator with a self-synchronous pipeline architecture.
- Post-layout simulation using Synopsys HSPICE® on a commercial 22nm bulk-CMOS process demonstrated $2.5\times$ enhancement in energy efficiency (174 TOPS/W) and $5\times$ improvement in area efficiency (2.01 TOPS/mm²) in comparison to the conventional accelerator [21].

The remainder of this paper is organized as follows: Section II introduces the MADDNESS algorithm and reviews prior LUT-based accelerators. Section III details the proposed all-digital MADDNESS accelerator, which incorporates self-synchronous pipeline accumulation. Section IV presents a performance evaluation based on post-layout simulations in a 22nm process and provides comparisons with conventional accelerators. Finally, Section V summarizes this paper.

II. BACKGROUND

MADDNESS[16] is a technique to reduce neural network inference costs by using approximate matrix multiplication (AMM) based on product quantization (PQ). It improves the inference accuracy after quantization by incorporating codebook generation into the training process of the neural network. This section provides an overview of the fundamental algorithms underlying PQ and MADDNESS, followed by a review of prior research on MADDNESS-based DNN accelerators.

A. Product Quantization

PQ is a dimensionality reduction technique that divides a high-dimensional vector space into smaller subspaces and quantizes each subspace independently [19]. To provide a clear illustration of PQ, consider the following example with a d -dimensional vector, $\mathbf{x} \in \mathbb{R}^d$. Initially, $\mathbf{x}$ is partitioned into M subvectors, resulting in:

$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_M], \quad \mathbf{x}_i \in \mathbb{R}^{d/M} \quad (1)$$

For each subvector $\mathbf{x}_i$, a separate codebook with K prototypes $\mathbf{C}_i = [\mathbf{c}_{i,1}, \mathbf{c}_{i,2}, \dots, \mathbf{c}_{i,K}]$ is prepared in advance. Subsequently, each subvector $\mathbf{x}_i$ is mapped to the corresponding prototype by using an encoding function, $enc_i(\mathbf{x}_i)$, yielding the quantized representation of $\mathbf{x}$ as follows:

$$\mathbf{q}(\mathbf{x}) = [enc_1(\mathbf{x}_1), enc_2(\mathbf{x}_2), \dots, enc_M(\mathbf{x}_M)] \quad (2)$$

The encoding function enc should be carefully designed as it influences the accuracy and efficiency of PQ. Specifically, it is crucial to design enc that has a low computational cost and does not degrade the inference accuracy of neural networks.

B. MADDNESS Algorithm

MADDNESS consists of two primary operations: encoding and decoding. Encoding corresponds to PQ, where the input is divided into subvectors, and the nearest prototype vector is identified for each subvector. In circuit implementations, encoding is equivalent to generating a one-hot LUT address. Decoding involves retrieving and accumulating the pre-computed multiplication results of the subvectors and synaptic weights from memory based on the generated address.

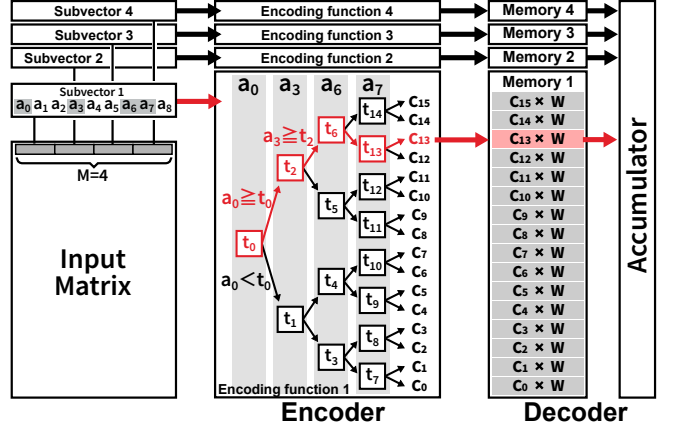


Fig. 1: Overview of the MADDNESS algorithm. The input vector is quantized into 16 prototypes by the decision tree classifier, and the corresponding multiplication results are loaded from memory to the accumulator.

Recent studies have proposed several MADDNESS-inspired algorithms. The main difference is the encoding function. The original MADDNESS proposed a balanced binary decision tree (BDT) based encoding function for fast and light inference. PECAN[17] employs Manhattan distance for encoding, while LUT-NN[18] utilizes Euclidean distance, to improve inference accuracy. Stella Nera[22] uses a matrix representation of the BDT to enable MADDNESS to be trained with back-propagation. In the remainder of this paper, the circuit implementations of these algorithms are collectively referred to as MADDNESS-based accelerators.

C. MADDNESS-based DNN Accelerator

The principal computational operation in MADDNESS accelerators is the calculation of an encoding function enc . Consequently, there has been a recent focus on the development of circuits capable of executing enc at high speeds with minimal energy consumption.

[21] proposed a method to efficiently compute enc as the similarity between the prototype and the input vector using time-domain analog circuit. The operational principle is as follows: firstly, the 6-bit precision input $\mathbf{x}$ and the 6-bit precision prototype $\mathbf{C}_i$ are converted into 60-bit thermometer codes. Subsequently, the circuit digital-to-time converter (DTC) is employed to calculate enc , whereby the Manhattan distance is converted into signal propagation delay. The DTC is constituted by a series of delay chains, with the k -th chain being responsible for computing similarity between input $\mathbf{x}$ and k -th prototype $\mathbf{C}_k$. The delay chain is constituted by delay cells, comprising variable delay elements and flip-flops connected in series. Each flip-flop pre-stores the prototype converted into thermometer codes. Therefore, by identifying the delay chain that provides the fastest signal propagation delay, it is possible to compute enc in an efficient manner. While this circuit demonstrates high energy efficiency through analog computing, the expansion of the codebook into thermometer codes necessitates a considerable allocation of area and memory capacity, with each n -bit codebook requiring 2^n -bit cells. Fur-

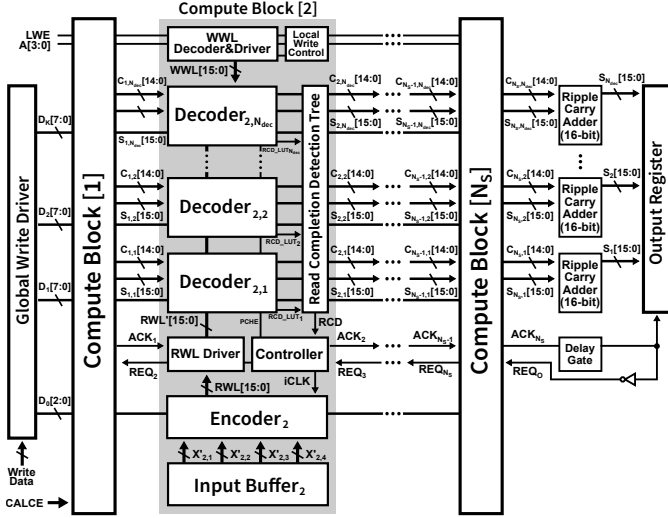


Fig. 2: Overall circuit of the proposed LUT-based DNN accelerator.

thermore, given the susceptibility of analog computing to PVT variations, the necessity arises for an additional circuit for post-fabrication calibration, with the objective of compensating for the computational errors that are caused by process variations.

To make the accelerator robust to PVT variation, [22] proposed a fully synthesizable all-digital accelerator featuring a decision tree-based classifier for PQ. Fig. 1 illustrates an example of the decision tree-based *enc* computation. A subvector (9-dimensional in the figure) extracted from the activation vector is classified into the corresponding prototype using a BDT. In this process, the elements of the subvector are compared with thresholds to determine the branch to be taken. In the example shown, the four elements, a_0 , a_3 , a_6 , and a_7 , are sequentially compared with the corresponding thresholds from left to right. Since there are four elements being compared in this example, the subvector is possible to be classified into $2^4 = 16$ prototypes. Based on the classification results, a lookup table is referenced to read out the precomputed inner product of the prototype and synaptic weights. This process is repeated for all four subvectors, and the results are accumulated to perform matrix multiplication. It has been reported that scaling the process node from 14nm to 3nm could achieve $4\times$ enhancement in energy efficiency and $32\times$ enhancement in area efficiency. However, the standard cell memory-based LUT consumes significantly higher energy and area than the standard SRAM-based LUT. Furthermore, the decision tree classifier requires additional memory costs for storing and reading threshold data, resulting in low energy efficiency.

This paper aims to improve energy efficiency by reducing the power consumption of both the PQ circuit and the LUT. The proposed all-digital macro incorporates a digital-CIM-based decision tree classifier for PQ and LUT based on area-efficient 10T-SRAM with an independent read port, achieving substantial reductions in energy consumption. Furthermore, since the proposed macro reliably operates in various PVT conditions by a global clock-free pipeline architecture, it can easily be implemented in advanced process nodes.

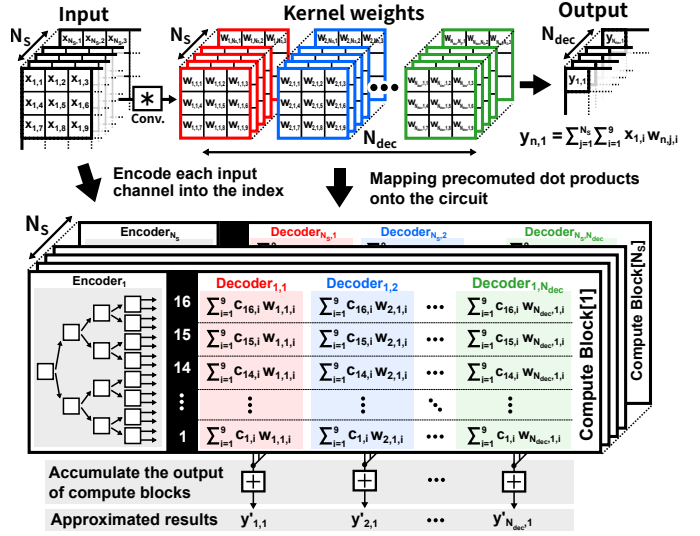


Fig. 3: CNN mapping onto the proposed circuit.

III. PROPOSED LUT-BASED DNN ACCELERATOR

A. Overall Architecture

Fig. 2 illustrates the overall macro of the proposed accelerator. The proposed macro consists of multiple serially connected compute blocks, a global write driver to update the contents of SRAM cells in the compute blocks, a 16-bit ripple carry adder (RCA), and an output register. Each compute block comprises an encoder that maps input vectors to the corresponding prototypes and multiple decoders that read precomputed inner product from SRAM based on the output of the encoder and perform accumulation, a self-synchronous pipeline controller for inter-block operations, a read wordline (RWL) driver, a read completion detection (RCD) tree for handshake control, an input buffer, a local write driver, a write wordline (WWL) decoder, and a WWL driver. The encoder includes a 4-level decision tree classifier to perform PQ on the input, while the decoder contains a 10T-SRAM array to store precomputed results, a 16-bit carry-save adder (CSA) for LUT output accumulation, and registers to hold the CSA results.

The proposed circuit has two adjustable parameters, N_s and N_{dec} . N_s is the number of pipeline stages, and N_{dec} is the number of decoders per compute block. In convolutional neural network[23] inference, N_s corresponds to the number of input channels that can be processed concurrently, while N_{dec} corresponds to the number of weight kernels that can be processed simultaneously, as shown in Fig. 3. Increasing either parameter can reduce the overhead of peripheral circuits per operation. Notably, increasing N_{dec} reduces the encoder overhead to $1/N_{dec}$, thereby enhancing area and energy efficiency. However, increasing N_{dec} also raises the WL wiring resistance within the LUT and the depth of the RCD tree, leading to increased latency. Additionally, since an internal critical path of each block limits block latency, worst-case delays may be increased for larger circuits. The optimal value for N_{dec} is evaluated in Section IV.

The operation of the compute block is as follows. First, prior to the inference, the precomputed dot products of the

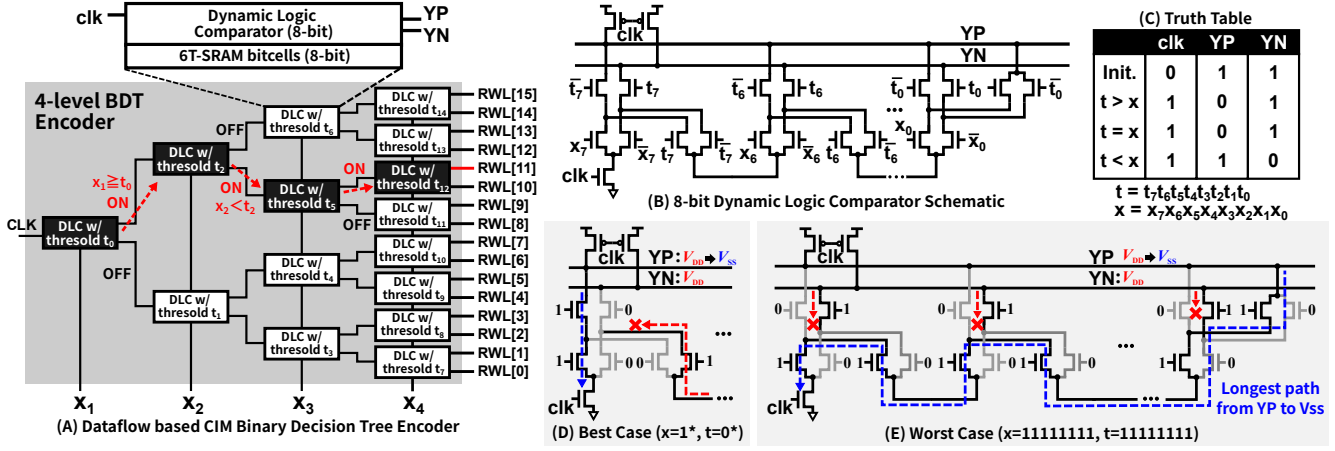


Fig. 4: Proposed decision tree-based encoder. (A) Overall schematic of the proposed encoder circuit. (B), (C) Schematic and truth table of the proposed dynamic logic comparator. (D), (E) Best and worst case operation of the DLC.

prototypes and synaptic weights are loaded in the corresponding SRAM cells embedded in decoders. In this paper, we employed an 8-bit integer precision. During the inference, the self-synchronous controller sets the block to an operation mode, which then activates the encoder to convert the subvector into the corresponding prototype. Similarly to [22], the proposed circuit employed a BDT for the encoding function, *enc*. Based on the mapped prototype, the decoder retrieves the precomputed dot product from the SRAM array. The retrieved data is accumulated with the output of the preceding block by the CSA, stored in the output latch of each decoder, and transmitted to the CSA of the subsequent block. Once all the lookup operations have been completed, the final summation is computed using the RCAs, and the result is stored in the output register.

Notably, all operations in the proposed circuit are performed asynchronously. In a typical clock-synchronized pipeline, the longest critical path among all stages determines the latency [24]. However, the asynchronous pipeline architecture allows each block to process the next input immediately, without waiting for the clock's rising edge, thereby minimizing latency [25]. Furthermore, this feature dynamically determines the operating speed, ensuring stable operation across various PVT conditions. A four-phase handshake protocol[26] is used for synchronization between compute blocks. Additionally, as will be detailed in Sec. III-B, the BDT inference engine also employs dynamic circuits, allowing only the necessary comparators to be activated automatically, which significantly reduces power consumption.

B. Decision Tree-based Encoder

As mentioned above, the proposed circuit uses a BDT in the encoder to classify subvectors into prototypes. Fig. 4(A) illustrates a block diagram of the encoder part of the proposed circuit. As shown, the encoder consists of dynamic logic comparators (DLCs) that compare the threshold values t with the elements of the subvector x . To classify the subvectors into one of 16 prototypes, the proposed encoder connects 15 DLCs in a tournament structure. Conventional clock-synchronous-

based circuit proposed in [22] require a global clock and many internal registers, resulting in high-power consumption. In contrast, the proposed encoder circuit achieves low-power computation through self-synchronous operation, eliminating both global clock and internal registers by employing dual-rail dynamic logic in the comparators. The red dotted line in Fig. 4(A) shows an example of how a subvector is classified into a prototype. First, all DLCs are precharged. Note here that each DLC stores a pre-learned threshold. Then, the root DLC is activated to compare the threshold t_0 with an element of the subvector x_1 . Depending on the result of the comparison, the next level DLC is selected to activate. By activating only the necessary DLCs, the circuit minimizes the discharge of precharged charges, thereby improving energy efficiency.

Fig. 4(B) and (C) show the schematic and the truth table of the DLC composed of a dual-rail dynamic logic. The DLC consists of eight 1-bit dynamic comparators connected in series. First, during the precharge phase ($\text{clk}=0$), both output nodes, YP and YN, are precharged to VDD. Additionally, the values of the two operands are set up in this phase. Then, in the evaluation phase ($\text{clk}=1$), the PMOS turns off, and depending on the input and threshold values, YP or YN is pulled down to VSS through the NMOS footer, as shown in the truth table. Note here that each 1-bit comparator discharges either YP or YN based on the comparison result if the comparison can be determined by the digit it is responsible for. If the comparison cannot be determined by its digit alone, the connection to the 1-bit comparator of the lower digit is activated, allowing YP/YN to be discharged based on the comparison of the lower digit. This method not only reduces the number of transistors required compared to comparators using naive subtractors, but also speeds up the comparison operation by immediately completing the calculation for those that can be determined by the higher digits alone, as shown in Figs. 4(D) and (E).

C. Two-port 10T-SRAM-based Decoder with CSA

After mapping the subvector to the corresponding prototype with the encoder, decoders retrieve the precomputed dot product

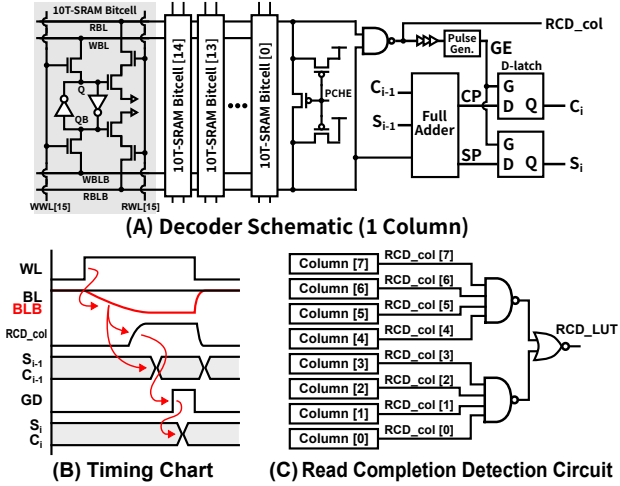


Fig. 5: 10T-SRAM based decoder with CSA unit. (A) Schematic of one SRAM column, (B) Timing chart of the decoding operation, (C) Schematic of the RCD circuit.

between the prototype and the synaptic weights from SRAM and perform accumulation. The proposed decoder consists of a 16-row, 8-column two-port 10T-SRAM array, a 16-bit CSA for accumulation, an output latch, and a 2NAND-1NOR circuit for RCD. This design facilitates high-speed data readout without the need for a sense amplifier (SA), as demonstrated in [10].

Fig. 5(A) and (B) illustrate the schematic and timing chart of a single column. The read operation proceeds as follows: first, based on the encoder output, one RWL in the column is asserted. Depending on the stored value, the selected SRAM cell fully discharges either RBL or RBLB. Subsequently, the full adder (FA) computes the carry and sum, and the output of the NAND gate (RCD_col) transitions to high. After a brief delay, the GE signal activates the latch, holding the FA outputs. When all RCD_col[i] signals in the LUT output high, they are combined into an overall RCD_LUT[i] signal via a NAND-NOR tournament, as depicted in Fig. 5(C). All RCD_LUT[i] signals in the same block are also aggregated into a single RCD[i] signal, as shown in Fig. 2, for the handshake control.

The proposed SRAM array incorporates a column-level RCD function to make the decoder robust to PVT variation. Traditional methods of estimating read latency often use a replica SRAM column, which accurately estimates read speed by mimicking the WL and BL layouts and conditions [27]. However, this approach is prone to column-to-column variability, requiring additional delay circuits for adjustment. In contrast, the proposed design features an independent RCD circuit for each column, enabling accurate detection even under high variability conditions. In addition, latch timing for the CSA outputs is generated simultaneously, effectively preventing setup violations over a wide range of PVT conditions.

IV. EVALUATION

The performance of the proposed circuit was evaluated via post-layout simulation using Synopsys HSPICE®, based on a layout implemented in a commercial 22nm bulk-CMOS process. The total chip area and core area of the macro were

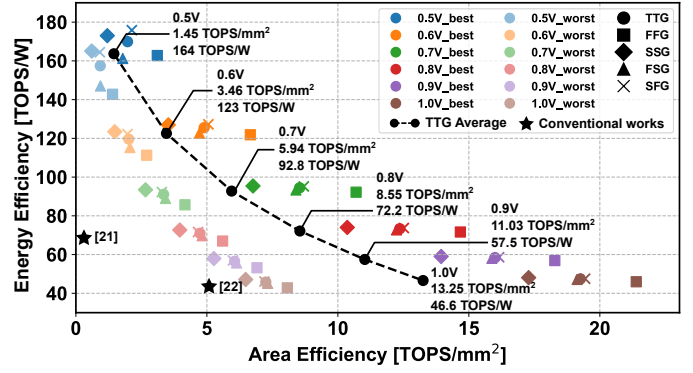


Fig. 6: Energy and area efficiency of the proposed circuit across various supply voltages and process corners. The black dashed line shows the average performance of the worst and best case at TTG corner. The upper right edge indicates better performance.

0.66 mm² and 0.20 mm², including 64kb SRAM ($N_{dec} = 16$, $N_S = 32$).

Fig. 6 evaluates the performance of the proposed circuit in terms of energy efficiency (TOPS/W) and area efficiency (TOPS/mm²) across various supply voltages (0.5V-1.0V) and process corners (TTG, FFG, SSG, SFG, FSG). Simulation settings are 25 °C, $N_{dec} = 4$, and $N_S = 4$. Since the latency of the BDT encoder varies depending on the input and BDT threshold values, as described in Sec. III-B, both the worst and best cases are shown in the figure. The black dashed line illustrates the average of the worst and best performances at the TTG corner. At 0.5V, the circuit achieves the highest energy efficiency of 164 TOPS/W, while at 1.0V, it demonstrates the highest area efficiency of 13.25 TOPS/mm² on average, highlighting a trade-off between energy consumption and throughput. The energy efficiency of the proposed circuit is determined mainly by supply voltage, which is nearly constant regardless of process corners and BDT encoder latency. Compared to conventional works (marked by black stars), the proposed circuit outperforms in both energy and area efficiency, particularly excelling in low-power scenarios.

Fig. 7 shows the energy, latency, and area breakdown for $N_{dec} = 4, 16$ and $N_S = 32$ at 0.5V. For energy, over 94 % of consumption for both $N_{dec} = 4$ and $N_{dec} = 16$ was attributed to the decoder, including the SRAM array and CSA. Similarly, the decoder accounted for a significant proportion (50 % to 80 %) of the area. This is because the energy and area of the decoder increase as $\mathcal{O}(N_{dec} \cdot N_S)$, while other components increase only as $\mathcal{O}(N_{dec})$ or $\mathcal{O}(N_S)$. In contrast, for latency, the encoder constituted the largest proportion over 40 % to 70 %. Comparing $N_{dec} = 4$ and $N_{dec} = 16$, the relative proportion of non-decoder components for energy and area decreases. On the other hand, the decoder's latency increased slightly due to the increased delay in the RCD.

To estimate the impact of N_{dec} values on performance more precisely, Table I shows the area and energy efficiency for $N_{dec} = 4, 8, 16$, and 32. Simulations were conducted under TTG corner at 25 °C, $N_S = 32$, and two supply voltage levels: 0.5 V for maximum energy efficiency and 0.8 V, the nominal

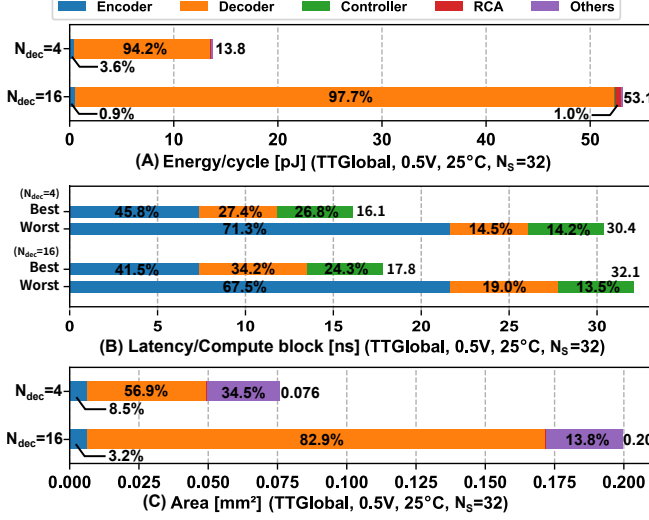


Fig. 7: (A) Energy, (B) latency, and (C) area breakdown based on post-layout simulation results.

TABLE I: Performance for different N_{dec}

Voltage	$N_{\text{dec}} = 4$	$N_{\text{dec}} = 8$	$N_{\text{dec}} = 16$	$N_{\text{dec}} = 32$
Energy efficiency [TOPS/W]				
0.5V	167.5	171.8 (+2.6%)	174.0 (+3.9%)	174.9 (+4.4%)
0.8V	73.0	74.4 (+1.0%)	75.1 (+1.0%)	75.4 (+1.0%)
Area efficiency [TOPS/mm²]				
0.5V	1.4	1.8 (+28.6%)	2.0 (+42.9%)	2.0 (+42.9%)
0.8V	8.7	10.8 (+24.1%)	11.3 (+29.9%)	11.5 (+32.2%)

* The numbers in parentheses indicate the improvement rate from $N_{\text{dec}} = 4$.

voltage of 22nm process. The figure indicates increasing N_{dec} improves both area and energy efficiency, with $N_{\text{dec}} = 32$ demonstrate 32.2 % to 42.9 % improvements in area efficiency and 1.0 % to 4.4 % improvements in energy efficiency compared to $N_{\text{dec}} = 4$. However, further increases in N_{dec} do not yield additional performance enhancements. For instance, the performance gain between $N_{\text{dec}} = 32$ and $N_{\text{dec}} = 16$ is 0 % to 2 %, which is almost negligible. Since larger N_{dec} values make the circuit vulnerable to local variations, $N_{\text{dec}} = 16$ is recommended to achieve an optimal balance between performance and reliability.

Finally, Table II provides a comparison of the proposed circuit ($N_{\text{dec}} = 16$, $N_{\text{S}} = 32$) with state-of-the-art works. Area efficiency was normalized to 22nm process for a fair comparison. For example, circuits implemented in a 65nm process were scaled by $(65/22)^2$ based on the reported data. However, since [21] employs analog computation in the encoder, area scaling was applied only to the digital parts. The simulation results show that the proposed circuit achieved $2.5\times$ higher energy efficiency and $5\times$ higher area efficiency at a supply voltage of 0.5V compared to [21]. Note that the performance of [21] does not include accumulator overhead. Compared to [22], the proposed circuit achieves $4.0\times$ the energy efficiency, but the area efficiency is 25 % lower. However, at a supply voltage of 0.8 V, the proposed circuit achieved $1.7\times$ increase in the energy efficiency and $4.2\times$ increase in the area efficiency, exceeding in both metrics. Instead of high energy efficiency, the proposed circuit has a lower operating frequency compared to

TABLE II: Comparison to prior accelerators.

	TCAS-I'23 [21]	arXiv'23 [22]	Proposed ($N_{\text{dec}}=16$, $N_{\text{S}}=32$)	
Measured /Simulated	Measured	Simulated	Simulated	
Operation Mode	MADDNESS (Analog)	MADDNESS (Digital)	MADDNESS (Digital)	
Process [nm]	65 (Planar)	14 (FinFET)	22 (Planar)	
Power Supply [V]	0.35/0.6/1.0 ⁽²⁾	0.55	0.5	0.8 ⁽¹⁾
Area [mm ²]	0.31	0.57	0.20	
Frequency [MHz]	77	624	31.2-56.2	144-353
LUT Precision	INT8 ⁽³⁾	INT8	INT8	
Throughput [TOPS]	0.089	2.9	0.28-0.51	1.33-3.26
Energy Efficiency [TOPS/W]	69	43.1	174	75.1
Area Efficiency [TOPS/mm ²]	0.29 (0.40) ⁽⁴⁾	5.1 (2.70) ⁽⁴⁾	2.01	11.34
ResNet9 Acc. (CIFAR-10)	89.0	92.6	92.6	92.6
Energy/op. [fJ] (Encoder)	7.47	1.27	0.054	0.11
Energy/op. [fJ] (Decoder)	7.02 ⁽⁵⁾	16.47	5.6	14.7

¹ Nominal voltage of the 22nm process. ² Multiple V_{DD} structure.

³ Adjustable between INT4 and INT32. ⁴ Scaled to 22nm process.

⁵ Accumulator is not included.

the [22], resulting in lower throughput. However, the smaller LUT layout of the proposed circuit allows it to maintain comparable area efficiency. Therefore, the proposed circuit can be applicable to situations where higher throughput is required by dividing the macros, although an additional adder is required. The performance improvement in energy efficiency is mainly attributed to the energy reduction in the decoder by employing an efficient 10T-SRAM array, which consumes 66 % less energy than standard cell memory-based LUT [22]. Additionally, the encoder reduced energy consumption by 95 % owing to eliminating the threshold readout cost and internal registers. It is also noteworthy that the classification accuracy on the CIFAR-10 dataset using ResNet9 was identical to that reported in [22].

V. CONCLUSION

This paper proposed a novel MADDNESS-based all-digital accelerator that significantly enhances energy and area efficiency for deep neural network inference. By employing a self-synchronous pipeline architecture, the design eliminates the need for a global clock, ensuring robust operation across various PVT conditions. The integration of a low-power binary decision tree encoder enables efficient PQ, while the implementation of an area-efficient lookup table using two-port 10T-SRAM substantially reduced energy consumption during decoding. Post-layout simulations using a commercial 22nm process demonstrated a $2.5\times$ improvement in energy efficiency (174 TOPS/W) and a $5\times$ increase in area efficiency (2.01 TOPS/mm²) compared to conventional accelerators.

ACKNOWLEDGEMENT

This work was supported by Japan Science and Technology Agency (JST), PRESTO Grant Number JPMJPR22B1 and JST BOOST, Grant Number JPMJBS2407, Japan.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017. DOI: 10.1145/3065386.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.
- [3] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Training Very Deep Networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015.
- [4] B. Steiner, M. Elhoushi, J. Kahn, and J. Hegarty, *OLLA: Optimizing the Lifetime and Location of Arrays to Reduce the Memory Usage of Neural Networks*, 2022. arXiv: 2210.12924 [cs.LG].
- [5] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. DOI: 10.1109/JSSC.2016.2616357.
- [6] J. Zhang, Z. Wang, and N. Verma, "In-Memory Computation of a Machine-Learning Classifier in a Standard 6T SRAM Array," *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 915–924, 2017. DOI: 10.1109/JSSC.2016.2642198.
- [7] H. Wang, R. Liu, R. Dorrance, D. Dasalukunte, D. Lake, and B. Carlton, "A Charge Domain SRAM Compute-in-Memory Macro With C-2C Ladder-Based 8-Bit MAC Unit in 22-nm FinFET Process for Edge Inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1037–1050, 2023. DOI: 10.1109/JSSC.2022.3232601.
- [8] K. Lee, S. Cheon, J. Jo, W. Choi, and J. Park, "A Charge-Sharing Based 8T SRAM In-Memory Computing for Edge DNN Acceleration," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 739–744. DOI: 10.1109/DAC18074.2021.9586103.
- [9] K. Yoshioka, "34.5 A 818-4094TOPS/W Capacitor-Reconfigured CIM Macro for Unified Acceleration of CNNs and Transformers," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, 2024, pp. 574–576. DOI: 10.1109/ISSCC49657.2024.10454489.
- [10] H. Fujiwara, H. Mori, W.-C. Zhao, *et al.*, "34.4 A 3nm, 32.5TOPS/W, 55.0TOPS/mm² and 3.78Mb/mm² Fully-Digital Compute-in-Memory Macro Supporting INT12 × INT12 with a Parallel-MAC Architecture and Foundry 6T-SRAM Bit Cell," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, 2024, pp. 572–574. DOI: 10.1109/ISSCC49657.2024.10454556.
- [11] H. Kim, J. Mu, C. Yu, T. T.-H. Kim, and B. Kim, "A 1-16b Reconfigurable 80Kb 7T SRAM-Based Digital Near-Memory Computing Macro for Processing Neural Networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 4, pp. 1580–1590, 2023. DOI: 10.1109/TCSI.2022.3232648.
- [12] M. Horowitz, "1.1 Computing's Energy Problem (and What We Can Do About It)," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323.
- [13] Y. Zhao, C. Li, X. Liu, R. Qian, R. Song, and X. Chen, "Patient-Specific Seizure Prediction via Adder Network and Supervised Contrastive Learning," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 30, pp. 1536–1547, 2022. DOI: 10.1109/TNSRE.2022.3180155.
- [14] Q. Deng, Y. Zhang, M. Zhang, and J. Yang, "LAcc: Exploiting Lookup Table-Based Fast and Accurate Vector Multiplication in DRAM-Based CNN Accelerator," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2019, pp. 1–6.
- [15] Y. Jeon, B. Park, S. J. Kwon, B. Kim, J. Yun, and D. Lee, "BiQGEMM: Matrix Multiplication with Lookup Table for Binary-Coding-Based Quantized DNNs," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC20)*, 2020, pp. 1–14. DOI: 10.1109/SC41405.2020.00099.
- [16] D. Blalock and J. Gutttag, *Multiplying Matrices Without Multiplying*, 2021. arXiv: 2106.10860 [cs.LG].
- [17] J. Ran, R. Lin, J. C. L. Li, J. Zhou, and N. Wong, *PECAN: A Product-Quantized Content Addressable Memory Network*, 2022. arXiv: 2208.13571 [cs.LG].
- [18] X. Tang, Y. Wang, T. Cao, *et al.*, "LUT-NN: Empower Efficient Neural Network Inference with Centroid Learning and Table Lookup," in *Proc. 29th Annu. Int. Conf. Mobile Computing Netw. (ACM MobiCom '23)*, 2023. DOI: 10.1145/3570361.3613285.
- [19] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011. DOI: 10.1109/TPAMI.2010.57.
- [20] C. McCarter and N. Dronen, *Look-ups are not (yet) all you need for deep learning inference*, 2022. arXiv: 2207.05808 [cs.LG].
- [21] H. Fuketa, "Lookup Table-Based Computing-in-Memory Macro Approximating Dot Products Without Multiplications for Energy-Efficient CNN Inference," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 10, pp. 3954–3963, 2023. DOI: 10.1109/TCSI.2023.3290151.
- [22] J. Schönleber, L. Cavigelli, R. Andri, M. Perotti, and L. Benini, *Stella Nera: Achieving 161 TOP/s/W with Multiplier-free DNN Acceleration based on Approximate Matrix Multiplication*, 2023. arXiv: 2311.10207 [cs.AR].
- [23] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791.
- [24] C. V. Ramamoorthy and H. F. Li, "Pipeline Architecture," *ACM Comput. Surv.*, vol. 9, no. 1, pp. 61–102, Mar. 1977. DOI: 10.1145/356683.356687.
- [25] S. Hauck, "Asynchronous design methodologies: an overview," *Proc. IEEE*, vol. 83, no. 1, pp. 69–93, 1995. DOI: 10.1109/5.362752.
- [26] V. W.-Y. Sit, C.-S. Choy, and C.-F. Chan, "A four-phase handshaking asynchronous static RAM design for self-timed systems," *IEEE J. Solid-State Circuits*, vol. 34, no. 1, pp. 90–96, 1999. DOI: 10.1109/4.736660.
- [27] B. Amrutur and M. Horowitz, "A Replica Technique for Wordline and Sense Control in Low-Power SRAMs," *IEEE J. Solid-State Circuits*, vol. 33, no. 8, pp. 1208–1219, 1998. DOI: 10.1109/4.705359.